

COMP232 Mock Exam — Model Answers (中英双语)

基于 PPT 和笔记。每题原文 + 翻译 + 答案。

Mock Q1(a) — 算法保密的安全性 (5 marks)

EN: *What are potential problems with the security protection based on the secrecy of an encryption algorithm?*

CN: 基于加密算法保密性的安全保护存在哪些潜在问题？

1. 算法难以保密 (Difficult to keep algorithm secret)

成功算法部署后必被逆向工程。一旦泄露，安全性全部丧失——密钥泄露只影响一把密钥，算法泄露全体遭殃。

Once deployed, a successful algorithm will eventually be reverse-engineered. When leaked, all security is lost — unlike a compromised key which only affects that one key, the algorithm cannot be easily replaced.

2. 隐藏缺陷发现太晚 (Hidden flaws discovered too late)

无公开审查，漏洞留在暗处。例：DVD 的 CSS (Content Scramble System) 算法，秘密设计，致命缺陷，发现时已部署数百万设备。

Without public scrutiny, vulnerabilities remain undiscovered. Example: CSS algorithm for DVD protection — kept secret, had a fatal flaw, discovered too late after millions of devices were deployed.

3. 违反 Kerckhoffs' Principle

安全性应依赖**密钥保密 (secrecy of the key)**，而非算法保密。密钥容易生成和替换，算法不能。

Security should depend on the secrecy of the **key**, NOT the secrecy of the algorithm. Keys are easy to produce and replace; algorithms are not.

PPT: COMP232-4 p7, COMP232-22 p6

Mock Q1(b) — 哈希碰撞改进攻击 (10 marks)

EN: *Suggest an improvement of this attack (finding hash collision), which would likely lead to faster discovery of the collision.*

CN: 原方法固定 I，随机选 J 直到 $H(J)=H(I)$ ——找特定目标碰撞，约需 2^n 次。请提出改进。

改进 (Improvement): 基于生日悖论 (Birthday Paradox) — 不找特定碰撞，而是找**任意两个不同输入**产生相同哈希值。

1. 随机选取大量不同输入 $M_1, M_2, M_3, \dots$
2. 计算每个 $H(M_i)$ ，储存 (哈希值, 输入) 对
3. 检查新算的哈希值是否与任意已有值相同
4. 找到任意一对 (M_i, M_j) 使 $H(M_i) = H(M_j) \rightarrow$ 碰撞

Why faster: 生日悖论 — 寻找任意碰撞只需约 $2^{n/2}$ 次。SHA-1 (160 位) 从 2^{160} 降到 2^{80} 。Google 2017 年 SHAttered 攻击即用此法造出两个不同但 SHA-1 相同的 PDF。

Birthday Paradox: finding **any** collision takes $\approx 2^{n/2}$ tries instead of 2^n . SHA-1 (160 bits) drops from 2^{160} to 2^{80} . Google's SHAttered (2017) used this to create two different PDFs with identical SHA-1.

Mock Q1(c) — DH 密钥交换 (10 marks)

EN: Describe the Diffie-Hellman (DH) Key Exchange algorithm and the attack on this algorithm that uses the computation of discrete logarithms. Why is DH considered secure?

CN: 描述 DH 密钥交换算法、基于离散对数计算的攻击，以及 DH 为何安全。

DH Algorithm:

公开参数 (Public): 大质数 P , 生成元 G (small constant, e.g. 5)

- Alice: 私钥 $a \rightarrow$ 计算 $A = G^a \bmod P \rightarrow$ 发送 A 给 Bob
- Bob: 私钥 $b \rightarrow$ 计算 $B = G^b \bmod P \rightarrow$ 发送 B 给 Alice
- Alice: $S = B^a \bmod P = G^{ab} \bmod P$
- Bob: $S = A^b \bmod P = G^{ab} \bmod P$
- Shared key $S = G^{ab} \bmod P$

Discrete Logarithm Attack:

攻击者看到 P, G, A, B 。要推导 S , 需从 $A = G^a \bmod P$ 反解 a (或从 B 解 b) ——这是离散对数问题 (Discrete Logarithm Problem)。

Why DH is secure:

正向 $G^a \bmod P$ 极快 (模幂 $O(\log a)$), 反向求 a 极其困难—— P 足够大 (如 2048 位) 时, 计算上不可行 (computationally infeasible)。安全性基于离散对数问题的计算困难性 (Computational Hardness)。

附：若问 RSA 如何答

EN: Describe the RSA encryption algorithm and the attack using integer factorization. Why is RSA considered secure?

RSA Algorithm:

- 选两大质数 p, q , $n = p \times q$, $\phi(n) = (p-1)(q-1)$
- 选 e ($\gcd(e, \phi(n)) = 1$), 计算 $d = e^{-1} \bmod \phi(n)$ (模逆元)
- 公钥 (e, n) , 私钥 (d, n)
- 加密: $C = m^e \bmod n$; 解密: $m = C^d \bmod n$

Integer Factorization Attack:

攻击者知道 (e, n) 。要求 d 需先求 $\phi(n) = (p-1)(q-1) \rightarrow$ 必须将 n 分解出 p, q 。这是大整数分解问题 (Integer Factorization Problem)。

Why RSA is secure:

$p \times q = n$ 极快, n 分解回 p, q 极其困难。 n 足够大 (如 2048 位) 时计算上不可行 (computationally infeasible)。安全性压在“知道 n 但求不出 p, q ”这一个点上。

Mock Q2(a) — 被动攻击 + 量子密码学 (8 marks)

EN: Define passive security attacks. Explain how the development of quantum cryptography may change the way of dealing with passive attacks.

CN: 定义被动安全攻击。解释量子密码学的发展如何改变应对被动攻击的方式。

Passive Attack (被动攻击):

攻击者截获/监听信息, **不影响系统资源或运行** (do not affect system resources or operation)。如窃听通信内容。特点: **难以检测 (hard to detect)**, 传统应对是**预防 (prevention)**——如加密。

Quantum Cryptography (BB84) changes the game:

传统网络: 被动攻击可静默复制信号 (silently copy signals)。

量子力学: “看”本身就是**测量 (Measurement)**, 测量必然**干扰量子态 (Quantum State)**。

BB84 协议中:

- Eve 窃听光子 → 必须测量 → 必须猜基 (+ 或 ×)
- 猜错 (50%) → 光子态被改变 → Bob 收到污染光子 → 结果出错
- Alice & Bob 公开对比少量 bit → 无 Eve: 错误率 $\approx 0\%$; 有 Eve: $\approx 25\%$ → 检测到窃听

根本变化 (Fundamental Change): 量子密码学将**不可检测的被动攻击 (Undetectable Passive Attack)** 转化为**可检测的主动干扰 (Detectable Active Interference)**。传统: 我们防着但不知道你在看。量子: 你看我必然知道, 然后这密钥我扔了。

PPT: COMP232-19-20-QM-26

Mock Q2(b) — RSA 部分同态 (7 marks)

EN: Explain why RSA can be considered as a partially-homomorphic encryption algorithm.

CN: 解释为什么 RSA 可被视为部分同态加密算法。

同态加密 (Homomorphic Encryption): 对密文运算 = 对明文对应运算, 无需解密。即

$$Enc(x) \cdot Enc(y) = Enc(xy)。$$

RSA satisfies multiplicative homomorphism:

$$Enc(m) = m^e \bmod n$$

$$Enc(m_1) \times Enc(m_2) = (m_1^e \bmod n) \times (m_2^e \bmod n) = (m_1 m_2)^e \bmod n = Enc(m_1 \times m_2)$$

The product of two ciphertexts equals the encryption of the product of plaintexts — **multiplying ciphertexts = multiplying plaintexts**, without ever decrypting.

RSA does NOT support addition ($Enc(m_1 + m_2) \neq Enc(m_1) + Enc(m_2)$), therefore it is **partially homomorphic (部分同态)** — only one type of operation (multiplication). For **Fully Homomorphic Encryption (FHE)**, both addition AND multiplication on ciphertexts must be supported.

Other PHE: Paillier supports additive homomorphism (can be used in voting).

PPT: COMP232-12-HomomorphicEnc-26

Mock Q2(c) — 错误哈希算法 (10 marks)

EN: What is wrong in the following algorithm for computing a hash function? Take a message M , generate a random private RSA key K . Encrypt M with K and take the first 240 bits of the result as a hash of M .

CN: 以下哈希算法有什么问题？

1. Not deterministic (非确定性)

哈希函数要求相同输入 $\rightarrow$ 相同输出。但每次**随机生成新密钥 K** $\rightarrow$ 同一 M 产生完全不同"哈希值"。连最基本的哈希函数定义都不满足。

A hash function must produce the same output for the same input. Random K each time breaks this.

2. Not one-way (非单向性 / No Preimage Resistance)

RSA 私钥加密 $= M^d \bmod n$ 。对应公钥 (e, n) 可解密: $(M^d)^e \bmod n = M$ 。攻击者用公钥可还原原文——哈希必须是单向的。

RSA encryption with private key can be reversed using the public key. Hash must be one-way.

3. Output length not fixed (输出长度不固定)

RSA 密文长度取决于模数 n ，而非固定输出。哈希函数必须产生**固定长度输出 (fixed-length output)**，如 SHA-256 始终 256 位。

Summary: A good hash function needs (1) Deterministic, (2) One-way (Preimage Resistance), (3) Fixed-length output, (4) Collision Resistance. This scheme fails all of them.

Mock Q3(a) — QRNG 优缺点 (5 marks)

EN: Explain what are the advantages and disadvantages of using quantum generator of random numbers in cryptography.

CN: 解释在密码学中使用量子随机数生成器的优缺点。

Advantages (优点):

- 根本性随机 (Fundamental Randomness):** 源于量子测量内在不确定性 (Quantum Uncertainty) — 叠加态坍缩是非确定性的 (Non-deterministic)。PRNG 是确定性算法，知道种子 (Seed) 即可预测。
- 不可预测 (Intrinsically Unpredictable):** 量子理论保证测量结果无法预先确定。
- Self-testing:** 部分 QRNG 支持自测试 — 无需信任硬件，设备自身可证明产生的是真量子随机数。

Disadvantages (缺点):

1. 更复杂、更昂贵 (More Complex & Expensive): 需要单光子源、精密探测器等专用硬件。
2. 主要用于高安全场景 (High-security Applications Only): 性价比低, 日常场景用经典方案 (Classical RNG) 即可。

PPT: COMP232-19-20-QM-26, Page 20-21

Mock Q3(b) — RSA 盲签名 (10 marks) ⚠ 不在 PPT

EN: Describe the RSA-based blind signature technique. Discuss its role in the implementation of electronic payment systems and electronic voting systems.

CN: 描述基于 RSA 的盲签名技术。讨论其在电子支付和电子投票中的作用。

Core Idea (核心思路):

Alice wants Bob to sign a message m **without Bob seeing m 's content**.

RSA Blind Signature Steps:

1. **Blinding (盲化):** Alice picks random r ($\gcd(r, n) = 1$), computes $m' = m \times r^e \bmod n$, sends m' to Bob
2. **Signing (签名):** Bob signs with his private key d :
 $s' = (m')^d = (m \times r^e)^d = m^d \times r^{ed} = m^d \times r \bmod n$ (since $ed \equiv 1 \pmod{\phi(n)}$, so $r^{ed} \equiv r$), returns s'
3. **Unblinding (去盲):** Alice computes $s = s' \times r^{-1} \bmod n = m^d \bmod n$ — gets Bob's standard RSA signature on m

Bob signed a blinded value — never saw the actual content, yet the signature is valid.

Applications:

Scenario	Role
E-Voting (电子投票)	Voter blinds ballot → Authority signs (verifying voter eligibility) → Voter unblinds → Submits signed ballot. Identity verification separated from vote content — authority confirms eligibility without knowing the vote
E-Payment (电子支付)	Buyer blinds e-coin serial → Bank signs (verifying balance) → Buyer unblinds → Pays merchant. Authorization separated from purchase content — bank confirms funds without knowing what was bought

Mock Q3(c) — ZKP + 应用 (10 marks)

EN: Explain what is a zero-knowledge proof (ZKP). What are the possible applications of ZKP?

CN: 解释什么是零知识证明 (ZKP) 及其可能应用。

Definition:

交互式证明协议。证明者 (Prover) 向验证者 (Verifier) 证明自己拥有某个秘密，但**不泄露秘密本身**。验证者除了"对方确实拥有秘密"外，什么都没学到。

An interactive proof where Prover convinces Verifier they know a secret **without revealing the secret itself**. Verifier learns nothing beyond the fact that Prover knows it.

Three Properties (三大性质):

Property	Meaning
Completeness (完备性)	If Prover has the solution, they can convince Verifier with high probability
Soundness (可靠性)	If Prover has no solution, they almost cannot cheat Verifier
Zero-Knowledge(ness) (零知识性)	Verifier learns nothing beyond the fact that Prover has a solution

Examples (PPT detailed):

- **Colour-blind Bob / Red-green card test:** Alice proves two cards are differently colored without revealing which is red/green
- **Graph 3-colorability:** Peggy proves she knows a 3-coloring of a graph, Victor learns no vertex color

Applications (应用):

- Zero-knowledge authentication (身份认证)
- Information exchange contracts (信息交换合同)
- Verifiable computing (可验证计算)
- Blockchain: zk-SNARK for private transactions (e.g. Zcash)
- Authenticity of edited photos (编辑照片真实性)
- Integrity of ML models (ML 模型完整性)

PPT: COMP232-16-ZKP-MPC-26

Mock Q4(a) — RSA 公因子攻击 (8 marks)

- EN: *What is the common factor attack on RSA and why is it practically possible?*
- CN: 什么是 RSA 公因子攻击？为什么在实践中可能发生？

Common Factor Attack (公因子攻击):

Two different users independently generate RSA keys. If their moduli share a prime factor (e.g., both used the same p):

$$\gcd(n_A, n_B) = p$$

Then $q_A = n_A/p, q_B = n_B/p$. Both keys are broken simultaneously. Euclid's algorithm computes gcd extremely fast ($O(\log n)$).

Why practically possible:

PPT: "Due to insufficiently good random number generators used in key generation, some number of keys used in the wild have common divisors."

Poor-quality RNGs (predictable seeds, insufficient entropy) → different devices may coincidentally generate the same prime. A 2012 study scanned real-world RSA public keys and found many pairs sharing common factors.

The attack is **not targeted** — attackers bulk-collect public keys and compute pairwise gcd, harvesting all weak keys at once.

PPT: COMP232-7-9-Algorithms-26, Page 15

Mock Q4(b) — 入侵检测两大类 (7 marks)

EN: What are the two main categories of intrusion detection methods? Describe advantages and disadvantages.

CN: 入侵检测方法的两个主要类别是什么？描述优缺点。

1. Signature-based (基于特征/知识, Knowledge-based):

Match against known attack patterns / signatures.

Advantages	Disadvantages
Low false alarm rate (低误报)	Cannot detect unknown/novel attacks (无法检测未知攻击)
Simple & efficient (简单高效)	Requires continuous signature database updates (需持续更新特征库)

2. Anomaly-based (基于异常/行为, Behaviour-based):

Learn normal behaviour baseline → flag deviations.

Advantages	Disadvantages
Can detect novel/unknown attacks (能检测未知攻击)	High false alarm rate (高误报)
Can identify privilege abuse (能发现权限滥用)	Needs continuous baseline maintenance (需持续维护基准)
Less OS-dependent (对操作系统依赖小)	High computational cost (计算开销高)

PPT: COMP232-13-Monitoring-ID-26, COMP232-14-TechniquesID-26

Mock Q4(c) — 协议漏洞分析 (10 marks)

EN: What is the possible issue with this protocol? How can it be fixed?

```
1 | 1. A → B: N_a
2 | 2. B → A: N_b
3 | 3. A → B: {k_ab, A, N_b}_{K_b}
4 | 4. B → A: {N_a}_{k_ab}
```

CN: 以下协议有什么问题？如何修复？

Issue (问题):

Msg 3 is encrypted with K_b (B's public key) but is **NOT signed**. Since K_b is public, **anyone can generate** $\{k, A, N_b\}_{K_b}$ claiming to be A. B decrypts successfully but cannot verify the sender's identity — an attacker I can impersonate A:

```
1 | I intercepts N_a, N_b → I sends {k_i, A, N_b}_{K_b} → B believes it's from A
```

Msg 3 缺少签名—— K_b 是公钥，任何人都能生成 Msg 3 冒充 A。B 无法区分。

Fix (修复):

A signs before encrypting with K_b :

$$\{\{k_{ab}, A, N_b\}_{sk_A}\}_{K_b}$$

B decrypts outer layer, then verifies A's signature with A's public key → confirms it was A who sent it.

Note: [8.3] 常见漏洞速查 表第 3 行：公钥加密但没有自己签名 → 先签名再加密。

Mock Q5(a) — CBC 模式 (7 marks)

EN: For a block cipher, what is the Cipher Block Chaining mode (CBC)? What is the main purpose of using CBC mode? What is the main disadvantage of this mode?

CN: 什么是 CBC 模式？主要目的？主要缺点？

What is CBC (CBC 是什么):

Each plaintext block is XORed with the **previous ciphertext block** before encryption. First block uses an **Initialization Vector (IV)**.

$$C_i = E_K(C_{i-1} \oplus P_i), \quad C_0 = IV$$

Main Purpose (主要目的):

Fix ECB's security flaw — in ECB, identical plaintext blocks produce identical ciphertext blocks, revealing data patterns. CBC's chaining makes identical plaintexts produce **different** ciphertexts.

Main Disadvantage (主要缺点):

Encryption cannot be parallelized — each block's encryption depends on the previous ciphertext. Must be sequential. (Decryption can be parallelized since C_{i-1} is already received.)

Mock Q5(b) — 多因子认证 (8 marks)

- EN: What is a multifactor authentication technique? Give an example and explain the rationale behind it.
- CN: 什么是多因子认证技术？举例并解释其原理。

Definition (定义):

Combining **two or more different categories** of authentication factors, rather than relying on a single one.

Factor	Category	Example
What you KNOW (你知道什么)	Knowledge	Password, PIN
What you HAVE (你拥有什么)	Possession	Token, bank card, mobile phone
What you ARE (你是什么)	Inherence	Fingerprint, iris, face

Example (例子): Bank card + PIN (银行卡 + PIN)

- Bank card (what you HAVE — Token)
- PIN (what you KNOW — Password)

Rationale (原理):

A single factor can be compromised — password guessed/leaked, token lost/copied. Combining factors means the attacker must defeat **two independent elements**: steal the card **AND** know the PIN. Leakage of one factor does not break authentication. Similarly: online banking password + SMS OTP — stolen password is useless without the phone.

Mock Q5(c) — 全同态加密 (10 marks)

- EN: What is Fully Homomorphic Encryption (FHE)? What is the main issue with applications of FHE? Give an example of an application which would benefit from using FHE.
- CN: 什么是全同态加密 (FHE)? 主要问题? 受益应用例子?

What is FHE (FHE 是什么):

Allows **arbitrary computation on ciphertexts** without decryption. Supports both operations:

$$Enc(x) \oplus Enc(y) = Enc(x + y), \quad Enc(x) \otimes Enc(y) = Enc(x \times y)$$

As long as both XOR and AND can be performed on ciphertexts, any Boolean circuit can be simulated → any program can run on encrypted data.
只要同时在密文上能做 XOR + AND，就能模拟任意布尔电路 → 运行任何程序。

First achieved by Craig Gentry (IBM, 2009), based on **Lattice Cryptography (格密码学)**, using **Bootstrapping** to control noise growth. Security based on LWE (Learning With Errors) → **quantum-resistant (抗量子)**.
RSA is partially homomorphic (multiplication only); Paillier (addition only). FHE = both.

Main Issue (主要问题): Performance (性能)

1,000× to 1,000,000× slower than plaintext. Reasons:

- Ciphertexts are huge polynomials carrying "information + noise"
- Noise grows with each operation; Bootstrapping resets noise but is extremely expensive
- Ciphertext and key sizes far larger than plaintext

Example Application (应用例子):**Cloud-based ML on encrypted medical data (云端加密医疗数据机器学习):**

Hospital uploads encrypted patient data to untrusted cloud → cloud runs ML training on ciphertexts → returns encrypted model. Cloud never sees any plaintext — confidentiality guaranteed by mathematics, not by trust in the cloud provider.